

O'REILLY®

第二版

Head First

设计模式

Head First
Design Patterns



[美] 埃里克·弗里曼(Eric Freeman) 著
[美] 伊丽莎白·罗布森(Elisabeth Robson)
& [美] 凯西·西拉(Kathy Sierra) [美] 贝特·贝茨(Bert Bates)
UMLChina 译

 中国电力出版社
CHINA ELECTRIC POWER PRESS



A Brain-Friendly Guide

Head First 设计模式

(第二版)

如果有一本设计模式书，比去看牙医更有趣，比税务局的表单更揭露真相，那该多好啊？这可能只是个幻想吧……



[美] 埃里克·弗里曼 (Eric Freeman) 著
[美] 伊丽莎白·罗布森 (Elisabeth Robson)

UMLChina 译

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

O'Reilly Media, Inc. 授权中国电力出版社出版



中国电力出版社
CHINA ELECTRIC POWER PRESS

Copyright © 2021 Eric Freeman and Elisabeth Robson. All right reserved.

Simplified Chinese Edition, jointly published by O'Reilly Media, Inc. and China Electric Power Press, 2022. Authorized translation of the English edition, 2022 O'Reilly Media, Inc., the owner of all rights to publish and sell the same.

All rights reserved including the rights of reproduction in whole or in part in any form.

英文原版由 O'Reilly Media, Inc. 出版 2021。

简体中文版由中国电力出版社出版 2022。英文原版的翻译得到 O'Reilly Media, Inc. 的授权。此简体中文版的出版和销售得到出版权和销售权的所有者——O'Reilly Media, Inc. 的许可。

版权所有，未得书面许可，本书的任何部分和全部不得以任何形式重制。

图书在版编目 (CIP) 数据

Head First设计模式：第二版 / (美) 埃里克·弗里曼 (Eric Freeman)，(美) 伊丽莎白·罗布森 (Elisabeth Robson) 著；UMLChina译. -- 北京：中国电力出版社，2022.2 (2022.3重印)

书名原文：Head First Design Patterns, 2nd Edition

ISBN 978-7-5198-5956-5

I. ①H… II. ①埃… ②伊… ③U… III. ①软件设计 IV. ①TP311.5

中国版本图书馆CIP数据核字(2021)第179142号

北京市版权局著作权合同登记 图字：01-2021-2211号

出版发行：中国电力出版社

地 址：北京市东城区北京站西街19号 (邮政编码100005)

网 址：<http://www.cepp.sgcc.com.cn>

责任编辑：刘 炽 (liuchi1030@163.com)

责任校对：王小鹏

装帧设计：Ellie Volckhausen, 张 健

责任印制：杨晓东

印 刷：北京天宇星印刷厂

版 次：2022年2月第一版

印 次：2022年3月北京第二次印刷

开 本：850毫米×980毫米 16开本

印 张：42

字 数：861千字

印 数：3001—6000册

定 价：128.00元

版 权 专 有 侵 权 必 究

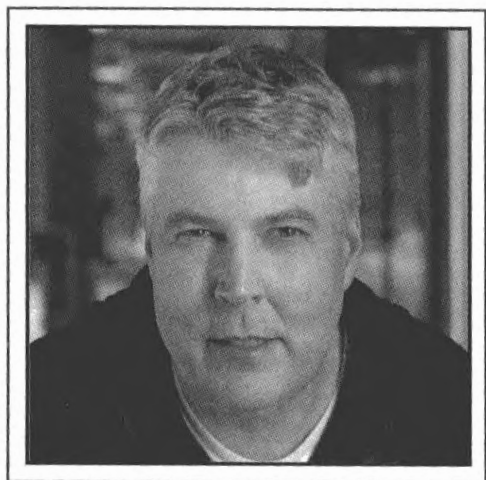
本书如有印装质量问题，我社营销中心负责退换

对四人组（GoF）来说，他们在捕获和沟通设计模式
方面的洞察力和专业性永久改变了软件设计的面貌，
改善了全世界开发人员的生活。

不过说实在的，什么时候我们能看到第二版？毕竟，
已经过了十年了。

二十五年

Head First设计模式的作者



←
Eric Freeman

Eric被Head First序列的共同创始人Kathy Sierra描述为“擅长来自潮人黑客、公司VP、工程师、智库等多个领域的语言、实践和文化的极少数个人之一。”

Eric是一名计算机科学家，在耶鲁大学获得博士学位。工作上，Eric是迪士尼公司Disney Online & Disney.com部门的前CTO。

Eric现在是Head First系列的联合总监，致力于制作WickedlySmart上的印刷和视频内容，这些内容被分发到业界领先的教育频道。

Eric的Head First主题包括《Head First设计模式》《Head First HTML & CSS》《Head First JavaScript Programming》《Head First HTML5 Programming》，以及《Head First Learn to Code》。

Eric住在得克萨斯的奥斯汀。



← Elisabeth Robson

Elisabeth是一名软件工程师、作家和培训师。从她是耶鲁大学的学生开始，她就对技术很狂热。在耶鲁大学，她获得了计算机科学硕士学位。

她目前是WickedlySmart共同创始人之一。在WickedlySmart，她创作了许多书籍、文章、视频，以及其他作品。在这之前，作为O'Reilly Media的Special Projects总监，Elisabeth制作了各种技术主题的现场公开课和在线课，把她的热情用于创造学习体验，以帮助人们理解技术。

不在电脑前的时间，你会发现Elisabeth在户外徒步、骑自行车、划船、园艺。相机总是在她身边。

Head First系列的创造者

Kathy Sierra



Bert Bates



在Virgin、MGM、Amblin'担任设计师和在UCLA担任新媒体写作老师时，**Kathy**就对学习理论感兴趣。她曾担任Sun Microsystems的Java高级培训师，并创立JavaRanch.com（现在是CodeRanch.com了），曾在2003年和2004年获得Jolt Cola Productivity 奖。

2015年，她因培养熟练用户和建造可持续社群获得电子前线（Electronic Frontier）基金的先锋奖。

Kathy当前关注的是前沿的运动科学以及技能习得教练，也称为生态动力学或者“Eco-D”。她用Eco-D来训练马，以更人道的方法来引领人马关系，给某些马带来了快乐（不幸，另外一些马受惊了）。那些幸运的（自主的）马因主人用了Kathy的方法，更快乐，更健康，比常规训练的小伙伴运动能力更强。

你可以在Instagram关注Kathy：

@pantherflows

在写书之前，**Bert**是一名开发人员，擅长老派的AI（大多为专家系统）、实时OS和复杂的调度系统。

2003年，Bert和Kathy编写了《Head First Java》并启动了Head First系列。从那时候起，他写了更多的Java书籍，并提供Sun Microsystems和Oracle的Java认证方面的咨询服务。他也培训了数以百计的作者和编辑，教他们如何做书和教课。

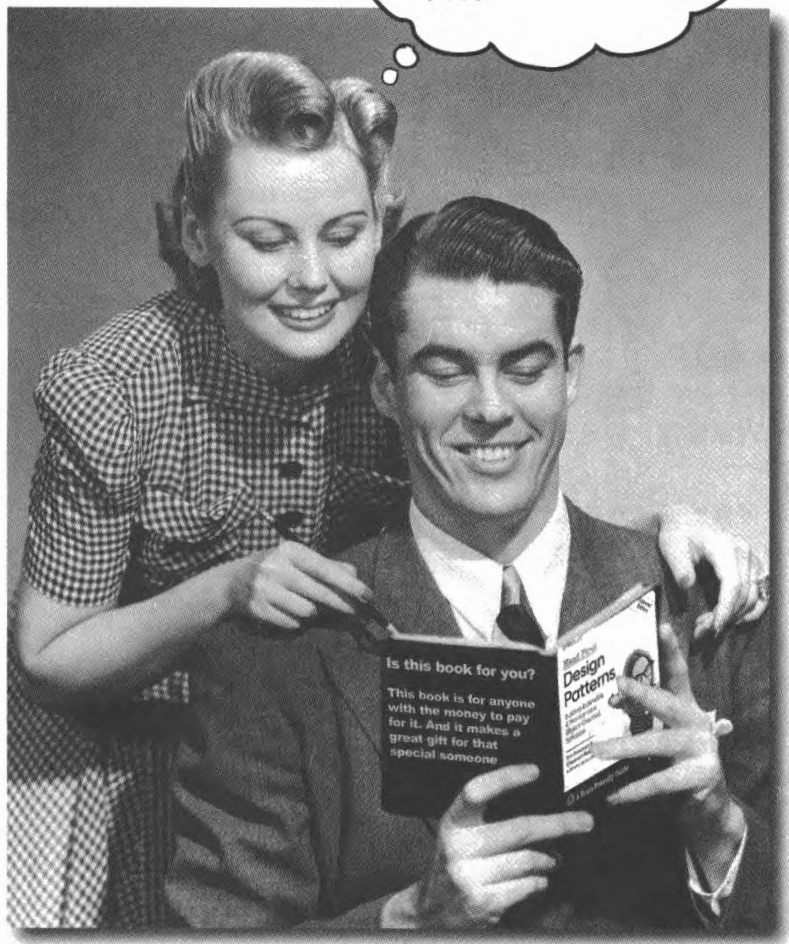
Bert是围棋玩家，2016年，他在惊恐和着迷中观看了AlphaGo击溃李世石的比赛。最近，他使用Eco-D（生态动力学）来改进他的高尔夫游戏和训练他的小鸚鵡Bokeh。

Bert和Kathy荣幸地认识Beth和Eric 16年了。Head First系列非常幸运，因为拥有他们作为关键的贡献者。

你可以在CodeRanch.com给Bert发消息。

如何使用本书 介绍

我无法相信，他们把这个也放进一本设计模式书里！



在本节，我们回答大家急着想知道答案的问题：
“那么，为什么他们把这放进一本设计模式书里？”

这本书是为谁写的？

如果你对所有这些问题都回答“是”：

- ① 你知道**Java**或另一种面向对象语言（不需要成为权威）？
- ② 你要**学习、理解、记住和应用**设计模式，包括设计模式所基于OO设计原则吗？
- ③ 你是不是更喜欢趣味盎然的**餐桌派对谈话**，而不是枯燥乏味的学术讲座？

本书就是为你写的。

我们的所有例子都是用Java写的，但如果你懂得的是另外一门面向对象语言，也应该能够理解本书的主要概念。

谁可能应该躲开这本书？

如果你对这些问题任何一个回答“是”：

- ① 对于**面向对象编程**，你是彻底的新手吗？
- ② 你是一个厉害的面向对象设计师/开发人员，正在寻找**参考书**？
- ③ 你是一位架构师，正在寻找**企业设计模式**？
- ④ 你是不是**害怕尝试新鲜事物**？你是不是宁可接受根管治疗，也不愿意接受条纹和格子混合的衣服？你是不是相信，如果面向对象概念拟人化了，技术书就不严谨了？

本书不是为你写的。



市场部备注：本书适合所有有信用卡的人。

我们知道你在想什么。

“这能算一本正经的编程书吗？”

“这一堆图是干什么的？”

“我确实可以这样学吗？”

我们也知道你的大脑在想什么

你的大脑渴求新奇。它总在搜寻、扫描、等待不寻常的事情。大脑这样构造，有助于让你活下来。

今天，你不太可能被老虎吃掉。然而你的大脑还是一直在留心。只不过你不知道。

你的大脑如何对待遇到的各种按部就班、普通、常规的事情？它能做的只是不让它们妨碍大脑真正的工作，记住那些确实重要的事情。它不会费心去记乏味的东西，通过一个“这显然不重要”的过滤器，把这些东西过滤掉了。

那么你的大脑怎么知道什么重要？假设你某一天外出徒步，突然一只老虎跳到你面前，你的大脑和身体里会发生什么呢？

神经元燃烧，情绪高涨，化学物质汹涌。你的大脑就是这样知道的……

这肯定重要！可别忘了！

不过，假如你在家或者在图书馆里，安全、温暖，没有老虎。你正在学习备考，或者尝试学一些比较难的技术，你的老板认为掌握技术需要一周时间，最多十天。

只有一个问题，你的大脑尝试给你帮个大忙。它尽量保证这些显然不重要的内容不要占用稀少的脑力资源。这些资源最好用来存储真正重要的事情，像大老虎，像火灾险情，像你怎样才不会再穿短衣裤滑雪。

没有简单的方式来告诉你的大脑，“嘿，大脑，非常感谢，不过不管这本书多么乏味，也不管登记的情感里氏震级多么低，但我真的希望你能把这些东西记下来。”

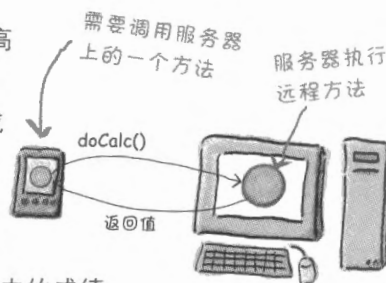


我们认为“Head First”的读者是学习者。

那么，怎么学习呢？首先，你要理解它，然后确保自己你不会忘记。这不是填鸭式往你的脑袋硬塞。根据认知科学、神经生物学和教育心理学的最新研究，学习不只是通过书本上的文字，还有很多东西。我们知道怎样激活你的大脑。

一些Head First学习原则：

可视化。和单独的文字相比，图片更容易让人记住，学习效率更高（对记忆和传递型的学习，提升高达89%）。而且图片让人更容易理解。把文字放进所关联图片中或放在旁边，比起放在底部或放在另一页，解决内容相关问题的效率高达两倍。



使用会话式和个人化风格。研究表明，如果用第一人称、会话的方式直接向读者讲述内容，而不是用正式的语调，学生课后考试中的成绩

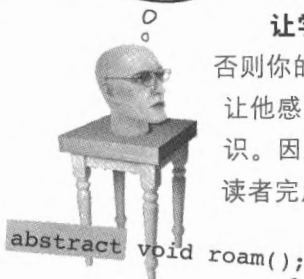
会提高40%。讲故事，而不是做报告。要用随意的语言，不要搞得太严肃了。一个是活力四射的派对伙伴，另一个是做报告的老师，这两个你会更注意哪一个？

作为一个抽象方法，
没有身体真是恶心。

让学习者想得更深。换句话说，除非你积极地活动你的神经元，

否则你的脑袋里什么也不会发生。必须激励读者，吸引读者，让他感到好奇和鼓舞，去解决问题，得出结论，并产生新知识。因此，需要一些挑战、练习和拓宽思路的问题，并要求读者完成一些活动，让左右脑和多种感觉都动起来。

说澡盆是一个浴室，说得通吗？
浴室是一个澡盆？或者是
“有一个”的关系？



没有方法体。
以分号结尾。

引起并保持读者的注意。我们可能都有这样的体验，“我真的想学会这个，不过我看了一页就想睡觉”。你的大脑注意的是那些超出常规、有趣、奇怪、抓人眼球、意料之外的东西。学习一项有难度的新技术，并不一定枯燥。如果学习过程不枯燥，大脑的学习速度会快很多。



触动读者的情绪。现在我们知道，记住东西的能力很大程度上依赖于情感内容。如果你关心某些东西，你就记得住。如果你感受到某些东西，你就记得住。不，我们不是在说心灵鸡汤。我们在说当你解决谜题，学习别人认为困难的东西，或者认识到“我比你更技术”的时候，泛起的惊讶、好奇、搞笑、骂街、唯我独尊等情感。



元认知：关于思考的思考

如果你真的要学，而且要学得更快，更深入，那么，把注意力集中在你怎样才能集中注意力，思考你是怎样思考的，学习你是怎样学习的。

在成长过程中，我们大多数没有上过有关元认知或学习理论的课程。我们应该学会，但是很少有人教我们怎么学会。

但我们假设，如果你手上有这本书，你真正要学会设计模式。你可能不想花很多时间，你还要记住你所阅读的并能够应用。为此，你必须理解其中内容。要最大程度理解本书，或者任何书籍或学习经验，就要让你的大脑担起责任，让它聚焦于这些内容。

诀窍是让你的大脑看到你正在学习的新材料确实，真的，很重要，和你的生命和健康紧密相关，像老虎出现在你面前一样。否则，你将陷入持续的拉锯战中，你的大脑会尽力阻止新内容扎根。

那么，怎样才能让你的大脑把设计模式看成和老虎一样重要呢？

有慢而乏味的方法，也有更快更高效的方法。慢的方法就是纯重复。你显然知道，如果你持续去啃某个东西，即使内容乏味至极，你也能学会并记住它。通过足够的重复，你的大脑就会说，“尽管感觉这对他不重要，不过既然他一而再、再而三地看同一个东西，我就假定这东西重要。”

更快的方法是做加速大脑活动的任何事情，特别是不同类型的大脑活动。上一页的内容就是解决方案的主要部分，而且是已经证实有助于你的大脑以对你有利的方式工作。例如，研究表明，把文字放在所描述图片里面（而不是放在页面的别处，比如作为标题或者放在正文中），会让你的大脑尝试去理解文字和图片之间有什么关系，而这就会让更多的神经元点火。更多的神经元点火=你的大脑有更多机会认为这些内容值得关注，而且可能需要记录。

会话式风格也有帮助，人们意识到自己在会话中时，注意力往往会更集中，因为他们想要跟上谈话的思路并发表恰当的言论。令人惊奇的是，你的大脑不必关心“会话”是发生在你和一本书之间！另一方面，如果写作风格正式而且干巴巴，你的大脑就会觉得像坐在一屋子人中间被动地听报告，没有必要保持清醒。

但图片和会话风格只是开始。



我们这样做：

我们使用图片，因为你的大脑处在图片频道，而不是文字频道。对你的大脑而言，一幅图顶得上1024个字。如果既有图片又有文字，我们把文字嵌进图片中，因为文字处在所描述的东西内部，大脑的工作效率更高，如果作为标题或埋在别的地方，效率就没那么高了。

我们使用冗余，用不同的方式，采用不同媒体和感官类型来讲述同一样东西，以便内容有更多机会留存在你大脑的更多区域中。

我们以意料之外的方式使用概念和图片，因为你的大脑喜欢新奇，我们使用至少带有一些情感内容的图片和点子，因为你的大脑会对情感的生物化学投入注意力。这会让你感觉到某些东西更有可能要被记住，即使这种感觉只不过是一小点幽默、惊奇和趣味。

我们使用个性化的会话风格，因为你的大脑相信你在交谈而不是被动听演讲时，会投入更多的注意力。即使在阅读时你的大脑也会这样做。

我们加入了90多个活动，因为与单纯的阅读相比，如果能确实做些事情，你的大脑会更乐于学习和记忆。我们提供的练习有一定挑战性，但还是可以做得出来，因为这正是大多数人所喜欢的。

我们使用多种学习风格，因为你可能更喜欢循序渐进地学习，而其他人可能希望先理解全貌，还有的人只是想要看代码案例。但不管你自己的学习偏好如何，每个人都会从多种方式表达同一内容中获益。

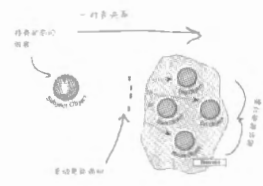
我们为你的大脑的两侧都准备了内容，因为你的大脑参与得越多，你就越有可能学会并记住，而且可以更长时间地保持关注。如果只有一侧大脑在工作，通常意味着另一侧有机会休息，你可以更有效率地学习更长时间。

我们会讲故事，留练习，从多个视角展示，因为如果强制让大脑做一些评价和判断，它就能更深入地学习。

我们提供了挑战，除了练习之外，还询问一些问题，这些问题的答案往往没有那么直接，因为在不得不工作在思考状态时，你的大脑更容易学会和记住东西。想想看，只是在健身房看别人流汗，是没法达到塑身的效果的。但我们尽力保证，你的工作努力用在了正确的事情上，这样你不会因为出现难以理解的例子、难以理解的术语或超短的文字而额外花费脑细胞。

我们使用人物。在故事中，在示例中，在图中等，因为，好吧，因为你是一个人。相对于东西而言，你的大脑会对人付出更多的注意力。

我们使用80/20方法。我们假设，如果你正在攻读软件设计的博士，这不会是你唯一的书。因此我们不会面面俱到，只提供你确实需要的东西。



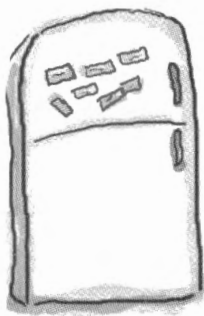
模式大师



重点

谜题





把这个剪下来贴在
冰箱上。

这些方法可以让你的大脑就范

那么，我们做了该做的，剩下的就要看你的了。这些提示只是开头，听一听你大脑的声音，弄清楚对你来说哪些做法可行，哪些做法不可行。做些新的尝试……

① 慢一点。你理解的越多，需要记的就越少。

不要只是阅读。停下来思考。当书中问你问题的时候，不要直接去翻答案。想象真的有人在问你问题。你强迫大脑想得越深，就越有机会学会并记住。

② 做练习。自己做笔记。

我们留了练习，但是如果有人帮你做了，那不就相当于有人帮你锻炼吗？不要只是看着练习发呆。把铅笔用上。有大量的证据证明，学习过程中如果实际动手，会提高学习效果。

③ 阅读“没有蠢问题”。

这意味着所有这部分内容。它们不是可有可无的花边，是核心内容的一部分，不要跳过！

④ 让本书成为上床之前你看的最后一本书，或者至少是最后一件有挑战性的事情。

学习的一部分（特别是转为长期记忆的部分）在你放下书之后发生。你的大脑需要有自己的时间来做更多的处理。如果在这段处理时间内，你又往大脑里灌输了新东西，那么你刚学的一些东西就会丢失。

⑤ 喝水。大量喝水。

沐浴在充足的液体中，你的大脑才能有最佳表现。缺水（可能你感觉口渴之前，就已经发生了）会削弱认知功能。

⑥ 说出来。大声说出来。

说话可以激活大脑的不同部分。如果你尝试理解某些东西，或者想提高后面记住它的机会，大声说出来。更好的办法是，大声地解释给别人听。这样你会学得更快，而且可能会有一些新的认识，这是光看不说时做不到的。

⑦ 倾听大脑的声音。

注意一下你的大脑是否超负荷。如果你发现自己开始流于表面，或者刚看的东西就忘记了，是时候休息一会了。一旦你达到某个点，如果还尝试往大脑里塞更多东西，也不会学得更快，甚至还可能损害正常的学习。

⑧ 感觉某些东西！

你的大脑需要知道这很重要。要真正融入到故事里。为照片加上你自己的标题。抱怨某个笑话很蹩脚，总比根本无动于衷要好。

⑨ 设计一些东西！

把它应用你正在设计的新东西中，或者重构旧项目。反正就是做一些事情，获取超越本书练习和活动的经验。你所需要的是一支铅笔和一个要解决的问题，一个可能受益于一个或多个设计模式的问题。

自述

这是一份学习体验，不是一本参考书。我们坚决排除挡在学习路上的所有东西，不管是什么。读完第一遍之后，你需要从头再读一次，因为本书对读者的背景知识做了一些假设。

我们使用简单的“类UML”图。

虽然这是你偶遇UML的好机会，但这不是本书覆盖的内容，也不是本书的前置知识。如果你以前从没见过UML，别担心，我们会一路告诉你一些UML的指引。因此，换句话说，你不必同时操心设计模式和UML。我们的图是“类UML”，虽然我们尝试严格遵循UML，但是出于我们自私的艺术家毛病，我们做了一些小改变。

我们没有覆盖每一个设计模式。

有许多设计模式：原始的基础模式(也称为GoF模式)、企业Java模式、架构模式、游戏设计模式，还有很多。但我们的目标是确保这本书比读者的体重更轻，因此我们不覆盖所有的模式。我们聚焦于原始GoF面向对象模式中重要的核心模式，并确保你真正、深入地理解如何和何时使用它们。其他（用到的可能性少得多的）模式我们会在附录中简要介绍。无论如何，一旦你读完《Head First设计模式》，你就能够拿起任何一本模式目录并很快学会。

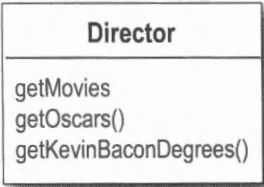
活动不是可有可无的。

练习和活动不是点缀，它们是本书核心内容的一部分。其中有一些有助于记忆，有一些有助于理解，有一些有助于应用所学知识。不要跳过这些练习。只有填字游戏谜题是你不必做的，但它们有益于给你的大脑一个机会从不同上下文来考虑这些词汇。

我们在普适的OO感觉上使用词汇“组合”，比严格的UML“组合”用法更有弹性。

当我们说“一个对象和另一个对象组合在一起”，意思是我们“有一个”（HAS-A）的关系。我们的用法遵循该术语的传统用法，GoF的文字中就是这样用的（后面你会学到）。最近，UML把这个术语精化为若干类型的组合。如果你是一名UML专家，你依然可以阅读本书，并且应该能够容易把组合的使用映射到你所阅读的更精细的术语。

我们用一个更简单的、修正版本的UML。



冗余是有意为之的,而且很重要。

Head First图书有一个与众不同的地方,就是我们希望你真真正掌握知识。我们希望你在学完这本书之后,能记住学过了什么。大多数参考书不会把保持和回想当作目标,但本书是关于学习的,因此你会看到一些同样的概念多次出现。

代码示例尽可能短小精悍。

读者告诉我们,如果为了寻找两行需要理解的代码需要在200行代码中跋涉,这是让人郁闷的。本书大多数例子尽可能在最小的上下文内展示,这样,你要学习的那部分内容就显得清晰而简单。别指望所有这些代码都很健壮,甚至都不要指望完整—例子是特地为学习编写的,功能不一定完整。

在某些例子中,我们没有包含所有需要的import语句,但我们假设,比如说,如果你是一名Java程序员,你应该知道ArrayList在java.util中。如果import不是正常核心JSE API的一部分,我们会提到。我们也把所有源代码放在web上,因此你可以从<http://wickedlysmart.com/head-first-design-patterns>下载。

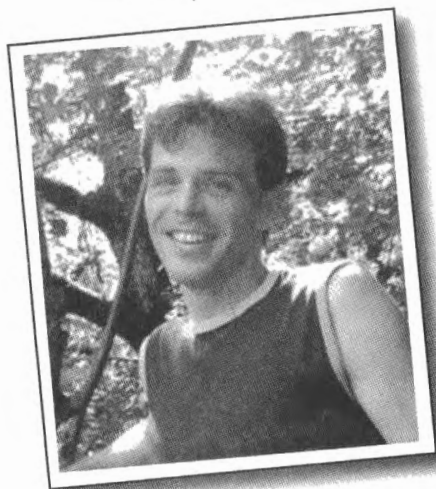
还有,为了聚焦于代码的学习方面,我们没有把类放进package中(换句话说,它们都在Java缺省的package中)。在现实世界中,我们不推荐这样做。你下载本书代码示例后,会发现所有类都在合适的package中。

脑力练习没有答案。

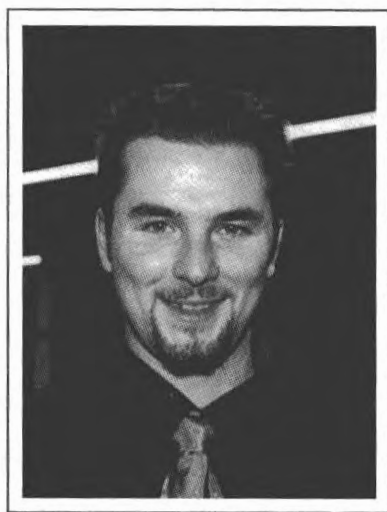
对其中某些练习来说,是没有正确答案的,对于其他练习,部分脑力活动的学习经验决定了你的答案是否正确,以及何时会正确。在某些脑力练习中,你会发现有暗示,指引你正确的方向。

技术审校

Jef Cumps



Valentin Crettaz



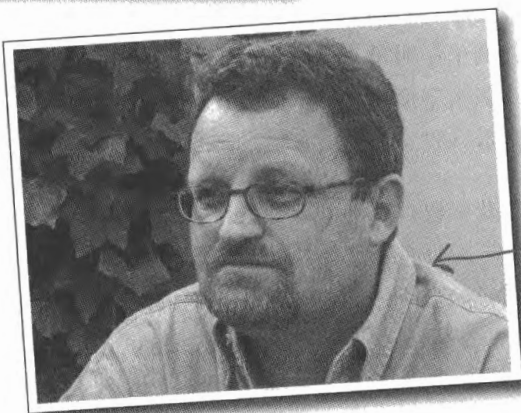
Barney Marispini



Ike Van Atta



HFDP极限审校
团队、无畏的
队长。



Johannes deJong



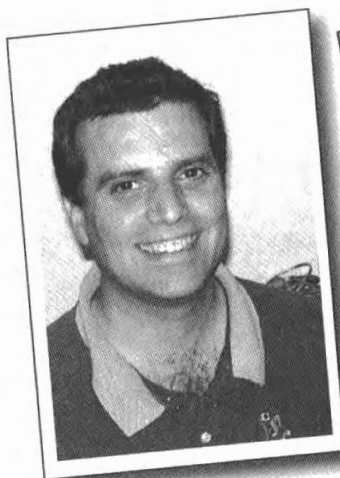
Jason Menard



怀念Philippe Maquet, 1960 - 2004。
你惊人的技术专长、不懈的热情、对学
习者的深切关心、将永远激励我们。



Philippe Maquet



Dirk Schreckmann



Mark Spritzler



技术审校，第二版



Julian Setiawan



George Heineman



David Powers



Trisha Gee

致谢

来自第一版

致O'Reilly:

我们对O'Reilly的Mike Llukides致以最大的谢意，感谢他开始这一切，并帮助把Head First概念形成一个系列。衷心感谢Head First背后的驱动力。首先要感谢的是Tim O'Reilly。感谢聪明的Head First“系列之母”Kyle Hart，“In Design之王”Ron Bilodeau，摇滚明星Ellie Volkhausen，感谢她灵感十足的封面设计，感谢Melanie Yarbrough在制作上的指引，感谢Colleen Gorman和Rachel Monaghan的硬核编辑，还有Bob Pfahler对索引的诸多改进。最后，感谢Mike Hendrickson和Meghan Blanchette支持本书并建立了团队。

致我们无畏的审校者:

我们特别感谢技术审校总监Johannes deJong。你是我们的英雄，Johannes。我们深深地感谢Javaranch审校团队共同管理者，已故的Philippe Maquet的贡献。你只手照亮数以千计开发人员生命，永远地影响他们（和我们）的生活。Jef Cumps特别善于从我们的草稿章节中发现问题，一次又一次地给本书带来了巨大的变化。感谢Jef！Valentin Cretazz（AOP人），他从第一本Head First书籍开始就和我们一起，（一如既往地）证明我们真的多么需要他的技术专长和洞察力。你真牛，Valentin。

HF审校团队有两位新人，Barney Marispini和Ike Van Atta，担任给本书挑毛病的工作，你们给出了一些真正关键的反馈。谢谢你们的加入。

我们还从Javaranch的主持人/权威Mark Spritzler、Jason Menard、Dirk Schreckmann、Thomas Paul和Margarita Isaeva那里获得了杰出的技术帮助。一如平常，特别感谢javaranch.com的老板Paul Wheaton。

感谢“挑选Head First设计模式封面”比赛的决赛入围者。获胜者Si Brewster，提交了他获胜的短文，说服我们选用本书封面的女人。其他决赛入围者包括Andrew Esse、Gian Franco Casula、Helen Crosbie、Pho Tek、Helen Thomas、Sateesh Kommineni和Jeff Fisher。

对于本书在2014年的更新，我们非常感谢以下技术审校：George Hoffer、Ted Hill、Todd Bartoszkiewicz、Sylvain Tenier、Scott Davidson、Kevin Ryan、Rich Ward、Mark Francis Jaeger、Mark Masse、Glenn Ray、Bayard Fetler、Paul Higgins、Matt Carpenter、Julia Williams、Matt McCullough和Mary Ann Belarmino。

致谢

来自第一版

致O'Reilly：

首先向Mary Treseler致以最高的谢意。她的超能力推动了所有事情，我们永远感激她为O'Reilly、Head First和作者们所做的一切。感谢Melissa Duffield和Michele Cronin为第二版排除了很多障碍。感谢Rachel Monaghan做了令人赞叹的编辑，让我们的文字冒出了新的火花。感谢Kristen Brown使得这本书不管是在线版本还是印刷版本看起来都这么漂亮。感谢Ellie Volckhausen 施展她的魔法，为第二版设计了一个全新的封面。感谢所有人！

致第二版的审校者：

感谢第二版的技术审校，在15年后重拾这个工作。感谢David Powers，我们的审校顾问（他是我们的，别想请他审校你的书），他不会错过任何事情。感谢George Heineman，他不仅提供详尽的评论、建议和反馈，还获得这个版本的技术MVP奖。感谢Trisha Gee和Julian Setiawan提供了宝贵的Java见解，这正是我们需要的，它帮助我们避免尴尬的Java错误。感谢所有人！

特别致谢

特别致谢Erich Gamma，他所做的远远超过评审本书（他甚至在度假时带着本书的草稿）。Erich，你对本书的兴趣激励了我们，你周到的审校给本书带来的提升不可估量。也感谢整个Gang of Four的支持和兴趣，并在对象村中做了一次特别的露面。我们也欠Ward Cunningham和模式社群一份情，他们创建了Portland Pattern Repository，为我们编写本书提供了不可缺少的资源。

非常感谢Mike Loukides、Mike Hendrickson和Meghan Blanchette。Mike L.一路都和我们在一起。Mike，你充满洞察力的反馈帮助本书成型，你的激励让我们一直前进。Mike H.，感谢你花了五年时间持久地推动我们写一本模式书。我们最终做到了并很高兴等待Head First。

写一本技术书需要很多人：Bill Pugh和Ken Arnold给我们提供了关于单件的专家意见。Joshua Marinacci提供了超棒的Swing技巧和建议。John Brewer的“Why a Duck?”论文激发了SimUDuck的产生（我们很高兴他也喜欢鸭子）。Dan Friedman激发了“小小单件”的例子。Daniel Steinberg扮演了“技术联络官”和情感支持网络的角色。感谢Apple的James Dempsey允许我们使用他的MVC歌曲。感谢Richard Warburton，确保这个版本中我们的Java 8代码更新符合标准。

最后，个人感谢Javaranch审校团队，感谢他们顶尖的审校和温暖的支持。你们的作用比你们所认为的还要大。

编写一本Head First书籍是一次荒野骑乘，幸好我们有两位向导：Kathy Sierra和Bert Bates。有了Kathy和Bert，你可以把所有写书的习惯抛在一边，进入一个充满故事、学习理论、认知科学和流行文化的世界，在那里，读者总是最大的。

目录（概览）

前言	xxv
1 欢迎来到设计模式：设计模式介绍	1
2 让你的对象保持消息灵通：观察者模式	37
3 装饰对象：装饰者模式	79
4 用OO的精髓做烘焙：工厂模式	109
5 独一无二的对象：单件模式	169
6 封装调用：命令模式	191
7 随遇而安：适配器和外观模式	237
8 封装算法：模板方法模式	277
9 管理良好的集合：迭代器和组合模式	317
10 事物的状态：状态模式	381
11 控制对象访问：代理模式	425
12 模式的模式：复合模式	493
13 现实世界中的模式：更好地与设计模式相处	563
14 附录：剩下的模式	597

详细目录

前言

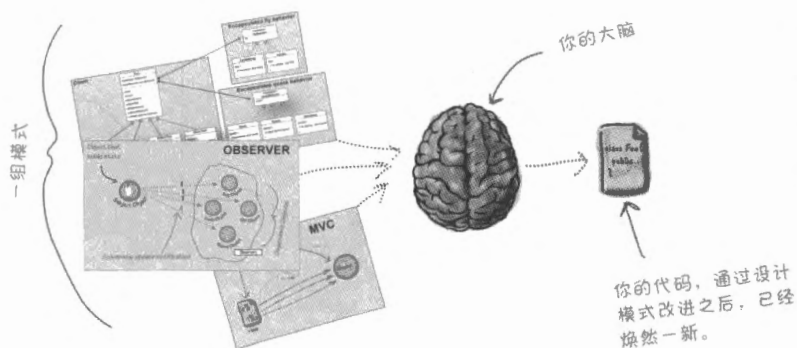
让大脑用于设计模式。 你在尝试学习一些东西，你的大脑却为了让学习不卡住而“好心帮忙”。大脑想，“把空间留给更重要的事情，例如躲避哪种野生动物，以及裸体滑雪是不是个坏主意。”因此，怎么欺骗你的大脑，让它认为懂得设计模式是一件性命攸关的事情？

这本书是为谁写的？	xxvi
我们知道你在想什么。	xxvii
我们也知道你的大脑在想什么	xxvii
我们认为“Head First”的读者是学习者。	xxviii
元认知：关于思考的思考	xxix
我们这样做：	xxx
这些方法可以让你的大脑就范	xxxi
自述	xxxii
技术审校	xxxiv
致谢	xxxv

设计模式介绍

欢迎来到设计模式

有人已经解决了你的问题。本章中，你将学到为何（以及如何）利用其他开发人员的智慧和教训。他们遭遇过同样的设计问题并且渡过了难关。本章结束前，我们会看看设计模式的用途和好处，以及一些关键的面向对象（OO）设计原则，并通过一个例子来了解模式是如何起作用的。使用模式的最佳方式是把它们装进大脑，在你的设计和已有的应用中识别用得上的地方。和代码复用不同，通过模式你获得了经验复用。



从简单的SimUDuck应用开始	2
但现在我们需要鸭子飞	3
但是，出大问题了……	4
Joe思考继承……	5
接口如何？	6
如果你是Joe，你会怎么做？	7
软件开发中唯一的不变	8
聚焦于问题……	9
分离变和不变	10
设计Duck的行为	11
实现Duck的行为	13
整合Duck的行为	15
测试Duck的代码	18
动态设置行为	20
封装行为的全貌	22
HAS-A比IS-A好	23
说起设计模式……	24
在当地餐厅偷听到……	26
在办公室隔间偷听到……	27
共享模式词汇的威力	28
我如何使用设计模式？	29
你的设计工具箱中的工具	32

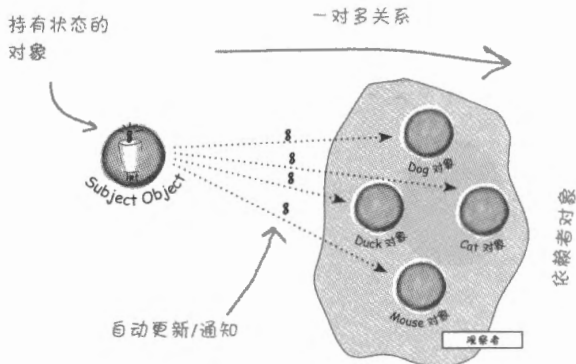
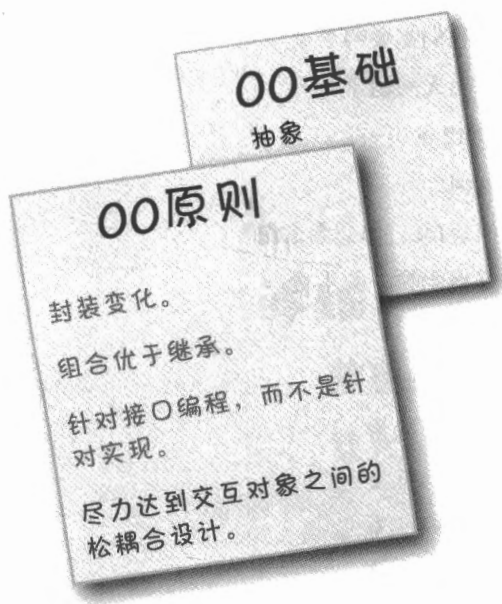
观察者模式

2

让你的对象保持消息灵通

你不想错过有趣的事情,对吧? 有一个模式可以让你的对象在发生某些事情时保持消息灵通。它就是观察者模式。它是最常用的设计模式之一,而且是不可思议的好用。我们打算介绍观察者的各种有趣的方面,像一对多关系以及松耦合。大脑里有了这些概念,还不能帮你在模式派对上露一手吗?

气象监测应用的概况	39
认识观察者模式	44
出版者 + 订阅者 = 观察者模式	45
定义观察者模式	51
松耦合的威力	54
设计气象站	57
实现气象站	58
启动气象站	61
在荒野中寻找观察者模式	65
编码生活改变应用	66
与此同时,回到Weather-O-Rama	69
试驾新代码	71
你的设计工具箱中的工具	72
设计原则挑战	73

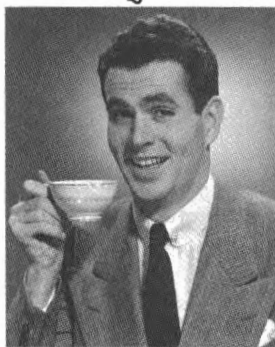


3 装饰者模式

装饰对象

本章可以称为“为爱用继承的家伙打开设计之眼”。我们即将再度探讨典型的继承滥用问题。你将学到如何在运行时使用对象组合的形式装饰你的类。为什么？一旦知道装饰的技巧，你将能给你的（或别人的）对象一个新的责任，而不需要对底层类的代码做任何改变。

我曾经以为真男人应该用继承处理一切。直到我领教到运行时扩展的威力比编译时扩展的威力要大。看看我现在的样子！

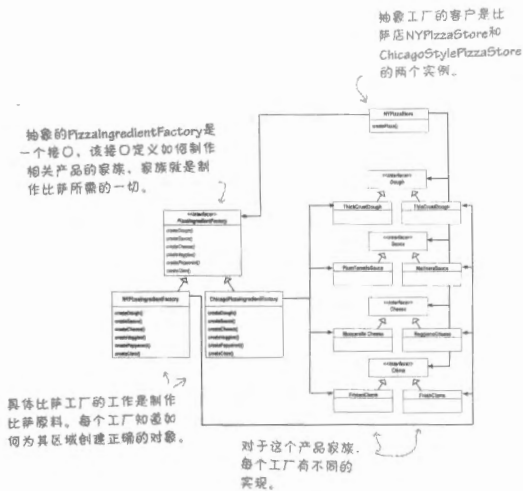


欢迎来到星巴兹咖啡	80
开放-关闭原则	86
结识装饰者模式	88
用装饰者构造饮料订单	89
装饰者模式定义	91
装饰我们的饮料	92
办公室隔间对话	93
新咖啡师训练	94
编写星巴兹代码	95
编写饮料代码	96
编写调料代码	97
提供一些咖啡	98
现实世界的装饰者：Java I/O	100
装饰java.io类	101
编写你自己的Java I/O装饰者	102
测试你的新Java I/O装饰者	103
你的设计工具箱中的工具	105

4 工厂模式

用OO的精髓做烘焙

准备烘焙一些松耦合OO设计。制造对象，不仅只有使用new操作符。你将会学习到，实例化这个活动不应该总是公开进行，因为经常会导致耦合问题。我们不希望这样，对吧？阅读下去，你会了解工厂模式如何能把你从令人难堪的依赖中拯救出来。



识别变化的方面	112
封装对象的创建	114
建立一个简单的比萨工厂	115
定义简单工厂	117
为比萨店而做的框架	120
允许子类决定	121
声明一个工厂方法	125
会见工厂方法模式的最后时刻到了	131
平行视角看创建者和产品	132
定义工厂方法模式	134
看看对象依赖	138
依赖倒置原则	139
应用原则	140
原料家族……	145
建造原料工厂	146
重做比萨……	149
重游比萨店	152
我们做了什么？	153
定义抽象工厂模式	156
比较工厂方法和抽象工厂	160
你的设计工具箱中的工具	162

5

单件模式

独一无二的对象

我们的下一站是单件模式,用来创建独一无二的对象,只能有一个实例的对象。你可能很高兴知道,单件模式的类图是所有模式中最简单的。事实上,图上只有一个类!但是不要掉以轻心,尽管从类设计的视角来说很简单,实现上还是需要一些深度的面向对象思想。因此,戴上思考帽,我们出发吧。



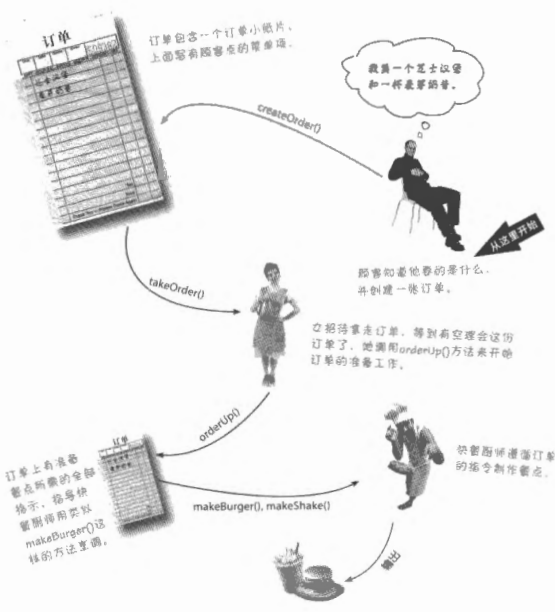
剖析经典的单件模式实现	173
巧克力工厂	175
定义单件模式	177
Hershey, PA 休斯顿, 我们有麻烦了……	178
处理多线程	180
我们能改进多线程吗?	181
与此同时, 回到巧克力工厂……	183
你的设计工具箱中的工具	186



6 命令模式

封装调用

本章中,我们把封装带到全新的高度:我们打算封装方法调用。没错,通过封装方法调用,我们可以把小块计算包装成形,以便调用计算的对象不需要操心如何做事情,只需使用我们包装成形的方法。有了这些封装好的方法调用,我们也可以聪明地做一些事情,像记录日志,或者复用它们以实现撤销(undo)功能。



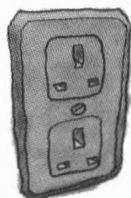
Home Automation or Bust	192
看看厂商类	194
命令模式简介	197
从餐馆到命令模式	201
我们的第一个命令对象	203
使用命令对象	204
分配命令到槽	209
实现遥控器	210
实现命令	211
逐步测试遥控器	212
是时候编写文档了……	215
我们要做什么?	217
是时候QA 撤销按钮了!	220
用状态实现撤销	221
添加撤销到吊扇命令	222
每个遥控器都需要“Party模式”!	225
使用宏命令	226
命令模式的更多用途: 请求队列	229
命令模式的更多用途: 日志请求	230
现实世界中的命令模式	231
你的设计工具箱中的工具	233

7 适配器 and 外观模式

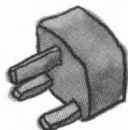
随遇而安

本章中,我们打算尝试不可能的任务,相当于把方形桩放入圆孔。听起来不可能?那是我们没有设计模式的时候。还记得装饰者模式吗?我们包装对象,赋予它们新的责任。现在,我们打算为不同的目的包装一些对象,使它们的接口看起来像别的东西。为什么我们要这样做?因为这样我们就可以把期望某个接口的设计适配到一个实现不同接口的类。不仅如此,我们还打算趁热打铁,探讨另一个包装对象以简化其接口的模式。

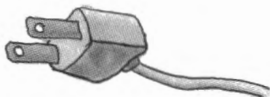
英式墙上插座



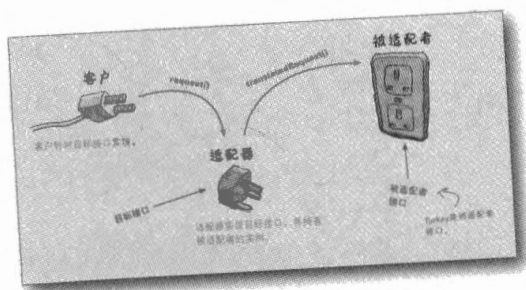
交流电适配器



美国标准的交流电插头



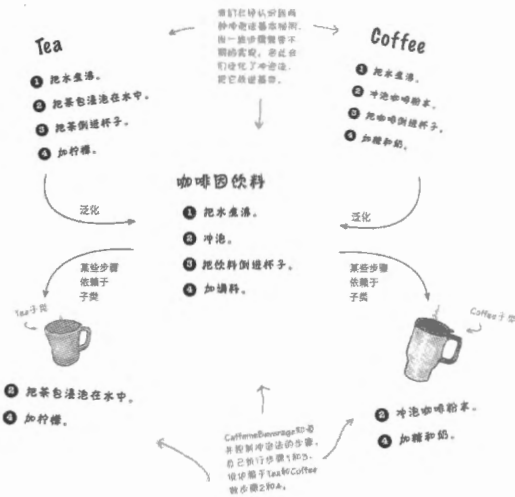
我们周围的适配器	238
面向对象适配器	239
如果它走路像鸭子,叫起来像鸭子,那么它一定可能是鸭子。被鸭子适配器包装的火鸡……	240
测试适配器	242
适配器模式解析	243
定义适配器模式	245
对象和类适配器	246
现实世界的适配器	250
适配Enumeration到Iterator	251
甜蜜的家庭影院	257
观赏电影(困难方式)	258
灯、相机、外观!	260
构造家庭影院外观	263
实现简化的接口	264
是时候观赏电影了(容易方式)	265
定义外观模式	266
最少知识原则	267
如何不赢得朋友和影响对象	268
外观模式和最少知识原则	271
你的设计工具箱中的工具	272



8 模板方法模式

封装算法

目前为止我们都在围着封装转；我们封装对象创建、方法调用、复杂接口、鸭子、比萨……接下来呢？ 我们打算深入到封装算法块，好让子类可以在任何时候，把自己挂接进计算中。我们甚至会在本章学到一个由好莱坞激发的设计原则。我们开始……



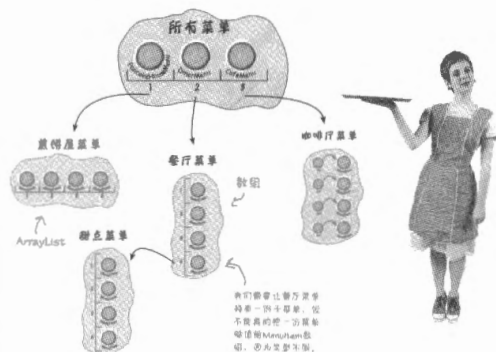
是时候来更多的咖啡因了	278
快速搞定一些咖啡和茶的类（用Java）	279
我们来抽象Coffee和Tea	282
进一步设计……	283
抽象prepareRecipe()	284
我们做了什么？	287
认识模板方法	288
模板方法带给我们什么？	290
定义模板方法模式	291
挂钩模板方法……	294
使用钩子	295
好莱坞原则和模板方法	299
荒野中的模板方法	301
用模板方法排序	302
有一些鸭子要排序……	303
比较鸭子和鸭子	304
我们来排序一些鸭子	305
鸭子排序机的制作	306
随Frame摇摆	308
用AbstractList定制列表	309
你的设计工具箱中的工具	313

9

迭代器和组合模式

管理良好的集合

有很多方法可以把对象堆起来成为集合。可以把它们放进数组、栈、列表、HashMap，随你挑。每一种都有自己的优劣，但在某些时候，你的客户想要遍历这些对象，而当他们这么做时，你希望把实现展示给客户吗？当然不！这太不专业了。好吧，你不必拿你的工作去冒险。本章中，你将看到如何能让客户遍历你的对象，而又无法窥视你是如何存储对象的。你也将学到如何创建某些对象超集合，一步跨越某些让人印象深刻的数据结构。除此之外，你还会学到一两件关于对象责任的事情。

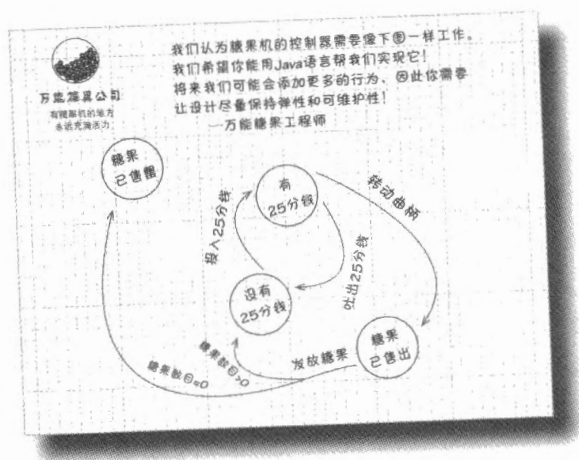


爆炸性新闻：对象村餐厅和对象村煎饼屋合并	318
检查菜单项	319
实现规格：我们的第一次尝试	323
可以封装遍历吗？	325
认识迭代器模式	327
给DinerMenu添加一个迭代器	328
用迭代器改写餐厅菜单	329
修正女招待的代码	330
测试我们的代码	331
回顾我们目前的设计……	333
用java.util.Iterator来清理	335
定义迭代器模式	338
迭代器模式的结构	339
单一责任原则	340
认识Java的Iterable接口	343
Java的enhanced for loop	344
看看咖啡厅的菜单	347
迭代器和集合	353
女招待准备好迎接精彩时刻了吗？	355
定义组合模式	360
用组合设计菜单	363
实现MenuComponent	364
实现MenuItem	365
实现组合菜单	366
现在轮到测试程序……	369
你的设计工具箱中的工具	376

10 状态模式

事物的状态

鲜为人知的事实： 策略和状态模式是双胞胎，刚出生就分开了。你以为他们的人生相似，但策略模式围绕可以互换的算法创建了极其成功的业务，而状态走的可能是更崇高的路，帮助对象通过改变它们的内部状态来控制其行为。不过你会发现，尽管路线不同，底层的设计是一样的。怎么回事？正如你将看到的，策略和状态意图差别很大。首先，我们来深入挖掘看看状态模式到底是什么，然后在本章末尾再回来探索它们的关系。



Java粉碎机	382
状态机101	384
编写代码	386
内部测试	388
你知道它会来的……变更请求！	390
乱糟糟的状态……	392
新的设计	394
定义State接口和类	395
重做糖果机	398
现在，我们来看看完整的GumballMachine类……	399
实现更多状态	400
状态模式定义	406
我们还是需要完成糖果机10中1游戏	409
完成游戏	410
为万能糖果公司的CEO演示	411
精神检查……	413
我们差点忘了！	416
你的设计工具箱中的工具	419



11

代理模式

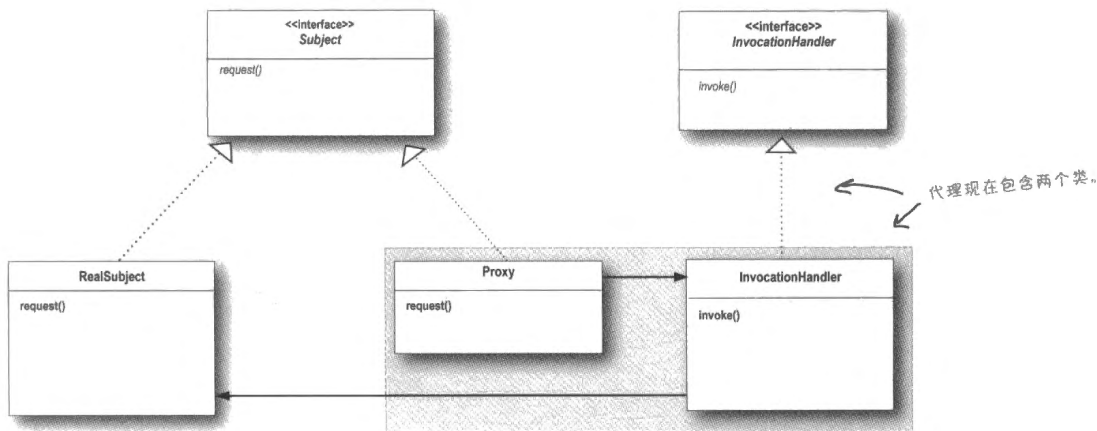
控制对象访问

玩过扮黑脸扮白脸的游戏吗？你是一个白脸，你提供服务的方式都是和蔼友善的，但是你不希望每个人都叫你做事，因此找了黑脸控制对你的访问。这就是代理要做的：控制和管理访问。就像你将看到的，挡在所代理的对象前面的代理有很多种方式。代理以通过Internet搬运被代理对象的整个方法调用而出名。另外一个出名的事情是，它耐心地守护某些很懒惰的对象。



极客配对

为监视器编码	427
测试监视器	428
远程方法101	433
让GumballMachine准备好成为一个远程服务	446
在RMI注册表中注册……	448
定义代理模式	455
轮到虚拟代理了（Virtual Proxy）	457
设计音乐专辑封面虚拟代理	459
编写Image Proxy	460
使用Java API的代理来创建一个保护代理	469
对象村极客配对	470
Person实现	471
五分钟短剧：保护主题	473
全貌：为Person创建动态代理	474
代理动物园	482
设计箱内的工具	485
音乐专辑封面浏览器的代码	489

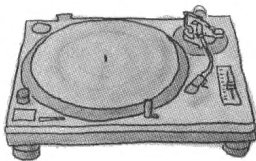
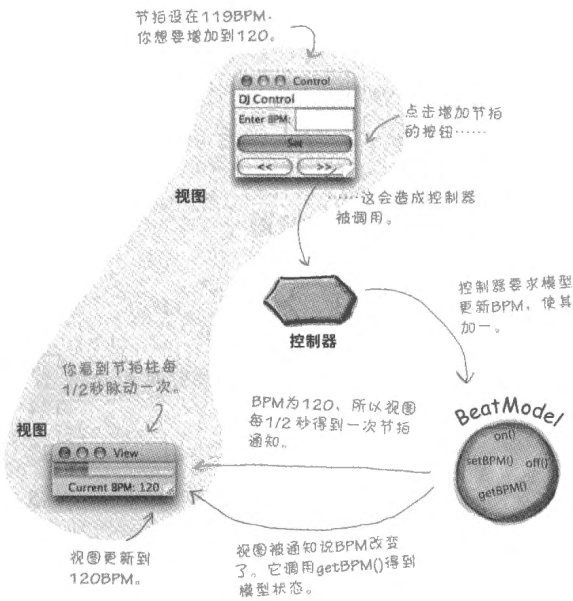


12

复合模式

模式的模式

谁能料到模式居然可以携手合作？你已经见识过炉边谈话的火爆场面（你们还没见识过模式死亡匹配的篇章呢，编辑逼着我们移除了）。谁能料到模式居然可以携手合作？好吧，信不信由你，有一些威力最强大的OO设计同时使用数个设计模式。准备好升级你的模式技巧，现在是复合模式时间。



携手合作	494
与鸭子重聚	495
我们做了什么？	517
一鸟一鸭瞰视图:类图	518
复合模式之王	520
认识模型-视图-控制器	523
靠近一点……	524
把MVC理解为一套模式	526
使用MVC控制节拍	528
建造碎片	531
现在来看看具体的BeatModel类	532
视图	533
实现视图	534
现在来看控制器	536
全部结合在一起……	538
探索策略	539
适配模型	540
运行测试程序……	542
设计工具箱内的工具	545

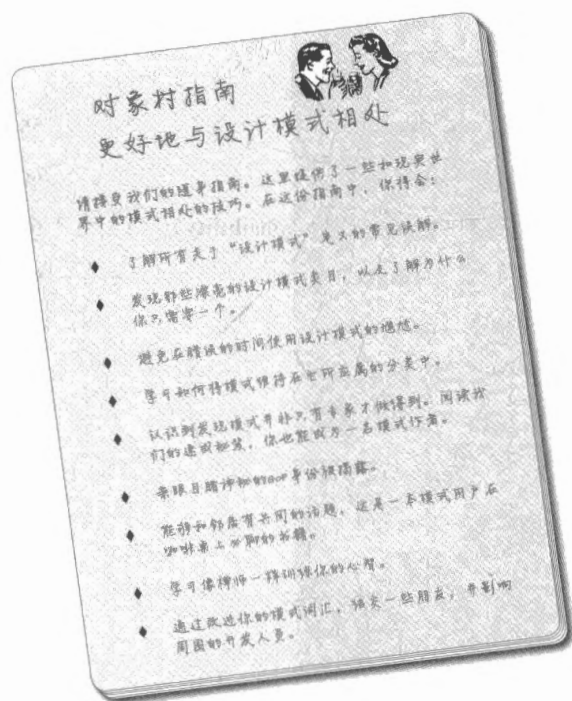
13

更好地与设计模式相处

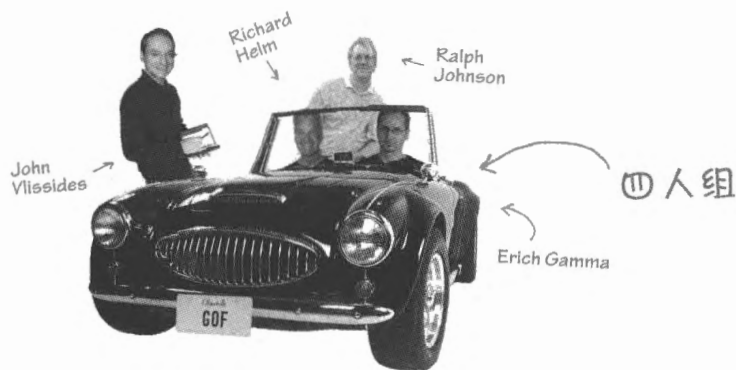
现实世界中的模式

啊哈!现在你已经准备好迎接一个充满设计模式的崭新世界。

但是,在你打开所有这些机会的新大门之前,我们需要谈到一些即将在现实世界中遇到的细节。没错,外面的世界比对象村要复杂一些。来吧!从下页开始,我们会指点你从对象村转到现实世界……



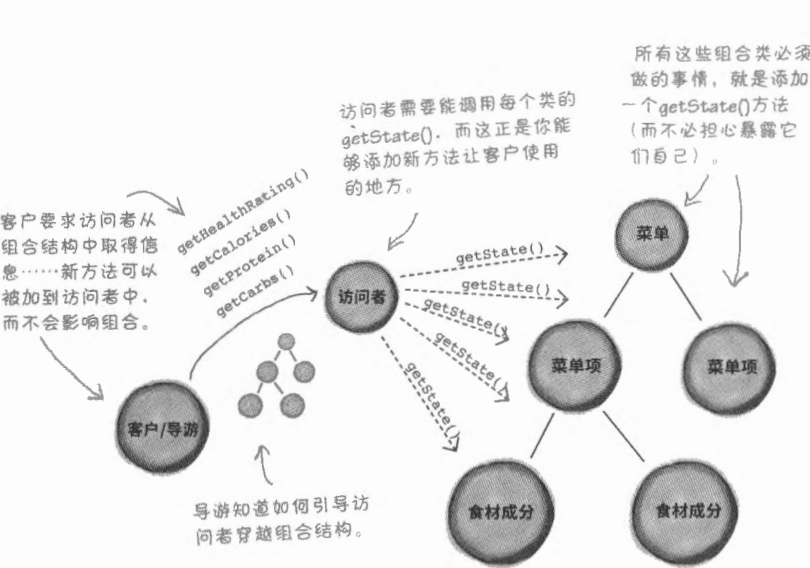
定义设计模式	565
更近地观察设计模式的定义	567
愿力与你同在	568
想当一个设计模式作者吗?	573
组织设计模式	575
用模式思考	580
模式的心智	583
别忘了共享词汇的威力	585
和GoF一起巡游对象村	587
你的旅途刚开始……	588
模式动物园	590
用反模式消灭恶势力	592
你的设计工具箱中的工具	594
离开对象村……	595



14

附录：剩下的模式

不是每个模式都广受欢迎。距离《Design Patterns: Elements of Reusable Object-Oriented Software》首次出版，已经过去了超过25年，很多事情已经变了。开发人员大量应用这些模式。我们在这个附录中所概括的模式，都是成熟、典型、正式的GoF模式，只不过可能不像我们之前所探索的模式那么经常被使用。但是这些模式也自有可取之处，如果你的情况刚好适用，你应该毫不犹豫地用上它们。这个附录的目标，是从概述的角度告诉你这些模式是关于什么的。



桥接 (Bridge)	598
生成器 (Builder)	600
责任链 (Chain of Responsibility)	602
蝇量 (Flyweight)	604
解释器 (Interpreter)	606
中介者 (Mediator)	608
备忘录 (Memento)	610
原型 (Prototype)	612
访问者 (Visitor)	614

i

索引